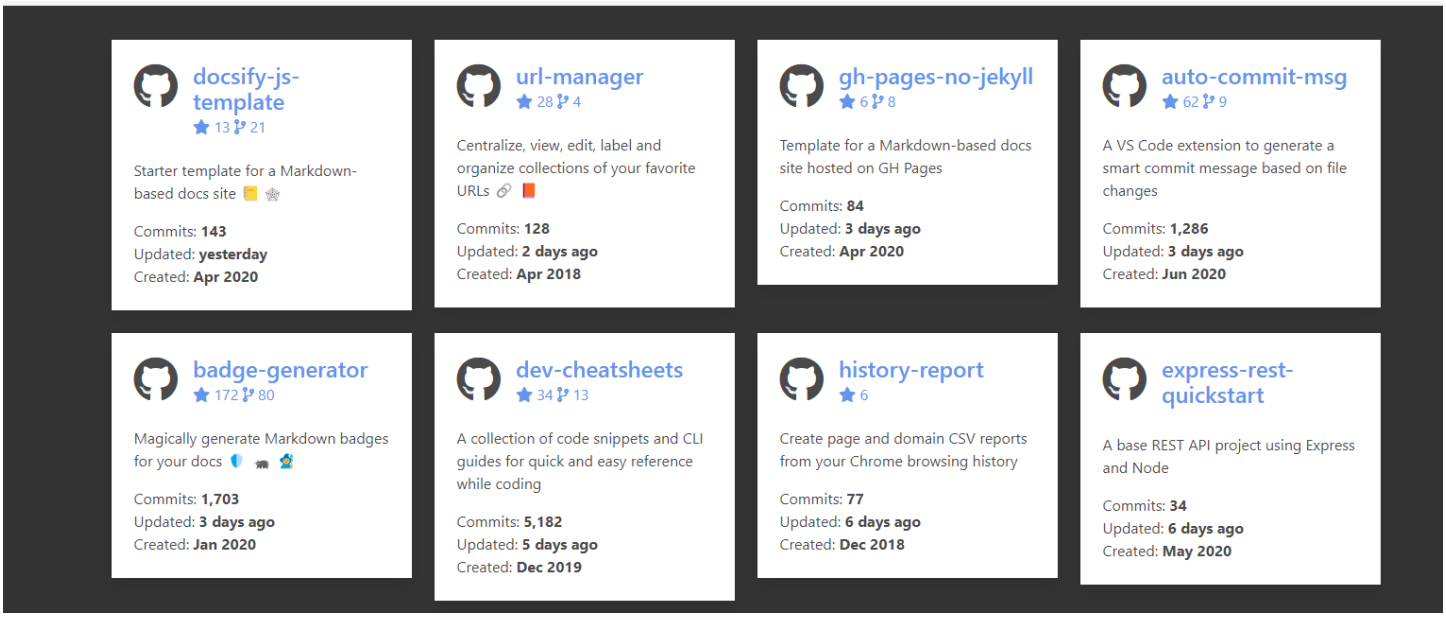
CSC300X: Web Development

Individual Assignment 5: GitHub Gallery

Due: Sunday, March 17, 2024

1. In this part of the assignment, you will practice using the fetch API by building a GitHub Gallery page similar to the one shown below using data from a user's GitHub repositories. The GitHub API is documented here: [GitHub API Documentation]



Each entry should include the repository name, description, creation date, update date, number of commits, list of languages, and the number of watchers. There should also be a hyperlink that will take you directly to the repository on GitHub. The GitHub icons can be found on Font Awesome. As a default, use your own GitHub profile for the gallery. Your page should also include a search field that allows for users to enter a GitHub username then display a gallery of repositories for that user.

Validate your HTML here: https://validator.w3.org/

Validate your CSS here: https://jigsaw.w3.org/css-validator/

To submit this part of the assignment, you will use your GitHub pages site again. In your *<username>.github.io* repository, create a subfolder for the homework and name it csc300x-hw5. All your work for this homework must be contained within this subfolder (HTML, CSS, JavaScript, images). Organize your files so they are easy to locate in subfolders as discussed in class. Do not directly use styles and scripts in the HTML file. All styles should be in .css files, and all scripts should be in .js files.

Be sure to edit the homepage for your GitHub Pages site to include a link to this subfolder, just like you did before.

When you push your changes to your github.io repo, your new sub-folder will also be deployed to your GitHub pages website.

Submit on Canvas: the link to the live deployed page.

For example, my link would be https://csc300x.github.io/csc300x-hw5

2. For the second part of the assignment, you will build a full-stack application with a Node/Express backend and a simple HTML/CSS/JavaScript frontend. We are not going to add this part to the GitHub pages site, because it only supports static websites, the Node/Express setup is not supported.

a. First, you will create a jokebook webservice with several endpoints. To begin, your webservice will be based on the following JavaScript objects.

```
let categories = ['funnyJoke', 'lameJoke'];
let funnyJoke = [
  {
      'joke': 'Why did the student eat his homework?',
      'response': 'Because the teacher told him it was a piece of cake!'
  },
  {
      'joke': 'What kind of tree fits in your hand?',
      'response': 'A palm tree'
  },
  {
      'joke': 'What is worse than raining cats and dogs?',
      'response': 'Hailing taxis'
  }
];
let lameJoke = [
  {
      'joke': 'Which bear is the most condescending?',
      'response': 'Pan-DUH'
  },
  {
      'joke': 'What would the Terminator be called in his retirement?',
      'response': 'The Exterminator'
  }
];
```

Create an express app with the following endpoints:

i. Jokebook Categories: Create a GET endpoint at /jokebook/categories that:

- responds with a list of possible categories in the jokebook.
  [funnyJoke, lameJoke]

ii. Jokes in a category: Create a GET endpoint at /jokebook/joke/:category that:

- responds with a JSON response.
- will send the list of jokes from the specified /:category
- add an optional query parameter 'limit' that limits the number of jokes returned.
- If the category is not valid, will respond with {'error': 'no category listed for [category]'} where [category] is replaced by the user-specified category.

iii. Add a new joke: Create a POST endpoint at /jokebook/joke/new that:

- Requires three parameters: category, joke, and response, sent with the body of the request.
- Should add the joke to the array corresponding to the given category, following the format: {joke: [JOKE], response: [RESPONSE]}, where [JOKE] and [RESPONSE] are the joke and response from the body of the request, respectively.
- Responds with the updated joke array.
- If missing one of the required parameters, or category isn't one of funnyJoke or lameJoke, the service should respond with {'error': 'invalid or insufficient user input'} and a 400 status code.

Use ThunderClient to test all the endpoints. Console.log the joke array to see if it has changed after a post request.

b. To finalize the webservice, create APIDOC.md, a file that provides the necessary information for anyone to use your API. For each endpoint, the documentation should include the request format, type, returned data format, description, an example request, example response, and error handling. Use the following template to create the markdown file.

```
# *jokebook* API Documentation
## Endpoint 1 - title here
**Request Format:**

**Request Type:**

**Returned Data Format**:

**Description:**

**Example Request:**

**Example Response:**
```

```
**Error Handling:**
```

c. Next, create the second part of the full stack project. Implement an HTML/CSS/JavaScript frontend that displays the jokes on a webpage. Style and appearance are up to you.
Serve the web files by using the public folder. An example full stack project would look like this:

```
example-node-project/
  .gitignore
  APIDOC.md
  app.js
  node_modules/
    ...
  package.json
  public/
    img/
      ...
    index.html
    index.js
    styles.css
```

The landing page should display a random joke from any of the above. Provide functionality for the user to view a list of all the joke categories by calling the first endpoint. Selecting or clicking on one of the returned categories should then call the second endpoint that gets the jokes of a category. This can also be done by directly inputting a category name on a search field.

Finally, add a form that lets a user create a new joke by calling the third endpoint. After this succeeds, display the jokes in that category to show it has been added.

EXTRA CREDIT: *If a user searches for a category that is not any of the above, add functionality to search for the category from the https://sv443.net/jokeapi/v2/ API. Search for a maximum of 3 two-part jokes for such a category. No inappropriate jokes at any point please. You will then add the new category and list of 3 jokes to your local jokebook array. You may need to find some CORS solutions.*

Follow the general guidelines outlined here https://uncg.instructure.com/courses/123038/pages/general-guidelines. For event listeners, favor using addEventListener, instead of 'onClick '. They are easier to manage.

To submit your work, please use a new GitHub repository, which you can name csc300x-hw5-2. Please do not upload your node modules to source code management. Submit a link to the new repo on Canvas.

For this assignment, you are allowed to work in pairs. You do not have to use term project teams. If you do work as a pair, please annotate your code with who worked on what. Additionally, both team members need to have independent submissions of the same code on their own repository.